

Characterizing Boolean Functional Synthesis via Knowledge Representation

Preey Shah, Aman Bansal, S Akshay, Supratik Chakraborty

Abstract

Boolean functional synthesis concerns synthesizing outputs as Boolean functions of inputs such that a relational specification between inputs and outputs is satisfied. This has several applications, including design of safe controllers for autonomous systems, certified QBF solving, cryptanalysis etc. Despite complexity-theoretic hardness results, several algorithms proposed in the literature are known to work well in practice. This motivates the investigation of whether there exist representations of input specifications that permit and also characterize efficient synthesis.

In this paper, we present a normal form called SAUNF that precisely characterizes tractable synthesis in the following sense: a specification is polynomial time synthesizable iff it can be compiled to SAUNF in polynomial time. Additionally, a specification admits a polynomial-sized functional solution iff there exists a semantically equivalent polynomial-sized SAUNF representation. SAUNF is exponentially more succinct than well-established normal forms like BDDs and DNNFs, used in the context of AI problems, and strictly subsumes other more recently proposed forms. It enjoys compositional properties that are similar to those of DNNF. Thus, SAUNF provides the right trade-off in knowledge representation for Boolean functional synthesis.

1. Introduction

Machine learning (ML) is increasingly being used for decision-making in safety-critical autonomous systems. As an example, self-driving cars use ML components to “understand” road traffic conditions. This “understanding” then feeds into decision-making for speeding up, slowing down or changing lanes. Clearly, a wrong decision can lead to catastrophic consequences. Ideally, only ML systems with formally proven safety guarantees should be used in such settings. Unfortunately, the complexity of ML systems that are amenable to automated formal verification significantly lags behind the complexity of ML systems that designers would like to deploy in autonomous systems. A pragmatic workaround is to use a monitor that checks in real-time if a safety-impacting decision, say D , produced

by an ML system satisfies a formal safety specification, say $\varphi_{safe}(D, E)$, where E represents key environment parameters (viz. speed, distance to neighbouring vehicle, etc). If $\varphi_{safe}(D, E)$ holds, actions (viz. speeding up) based on D can be safely taken. Otherwise, we must replace D by a variant D' that satisfies $\varphi_{safe}(D', E)$, while remaining “close enough” to D . Since D' must be generated quickly in systems with fast dynamics, we require an efficient circuit/program that takes D and E as inputs and outputs D' such that $\varphi_{safe}(D', E) \wedge (dist(D, D') \leq \varepsilon)$ holds for a suitably defined distance metric $dist$ for decisions, and a threshold ε . Note that it is often much simpler to specify the relation between D, D' and E (as given above) than to express D' as an explicit function of D and E . If D, D' and E are represented as Boolean vectors, the above problem translates to Boolean functional synthesis – a topic that has seen a revival of interest in recent years.

Interestingly, the above problem is also useful for “inverting” binarized neural networks, wherein we know the precise relation between outputs O and inputs I of a neural network, and are interested in asking what inputs I would cause the network to output O . This is useful in understanding, for example, what a trained neural network “thinks” is a “bird” or “car”.

Indeed, the core computational problem in such settings is to synthesize Boolean outputs (viz., D') as circuits (or loop-free programs) of Boolean inputs (viz. D, E) such that a given relational specification between inputs and outputs is satisfied. Several algorithmic approaches tackle this problem with varying levels of success (see related work). In this paper however, we are interested in viewing the problem through the lens of Knowledge Representation. That is, what would be the normal form representation of the input specification which could *characterize* polynomial time (and size) solutions? Our primary contributions are:

- We present SAUNF, a new knowledge representation form that characterizes poly-time Boolean function synthesis, and analyze various operations on it.
- We show that SAUNF is (often exponentially) more succinct and subsumes several other sub-classes (viz. dNNF, ROBDD, SynNNF). We also present SAUNF

representations of variants of the well-known *factorization* problem exhibiting its further utility.

- We present a novel algorithm for compiling a Boolean relational specification in CNF to SAUNF.

Related Work: Recent work on Boolean functional synthesis has mostly been towards design of efficient algorithms (Rabe and Tentrup 2015; Marijn Heule and Biere 2014; Rabe and Seshia 2016; Jiang and Balabanov 2011; Fried, Tabajara, and Vardi 2016; Akshay et al. 2017; Chakraborty et al. 2018; John et al. 2015; Kuncak et al. 2010; Akshay et al. 2018; Golia, Roy, and Meel 2020), which synthesize outputs as functions of inputs. All these algorithms, using techniques ranging from CEGAR to decision tree learning, empirically work well on some large collections of benchmarks. However, unless some complexity theoretic conjectures are falsified, Boolean functional synthesis requires super-polynomial space and time (Akshay et al. 2018), prompting the search for sufficient conditions for efficient synthesis, e.g., wDNNF and ROBDDs give quadratic-time algorithms for synthesis but do not characterize it.

Knowledge representations of Boolean functions have been investigated extensively over the last decades (Cadoli and Donini 1997; Darwiche 2001a; Muise et al. 2016; Darwiche and Marquis 2011) resulting in normal forms with many interesting properties. For Boolean Functional Synthesis, the related question of existential quantification (or forgetting) (Lang, Liberatore, and Marquis 2011; Fried, Tabajara, and Vardi 2016) has been considered, but they work for all orders in which variables are quantified and hence are (exponentially) more restrictive than necessary. A recent work (Akshay et al. 2019) provides a normal form called SynNNF which only synthesizes a very specific type of (Skolem) function and hence does not provide a characterization of efficient synthesis itself. As we shall see, our normal form strictly subsumes SynNNF.

2. Preliminaries

Let V be a finite set of Boolean variables. A *literal* ℓ over V is either v or $\neg v$, where $v \in V$. Given a Boolean function $F : V \rightarrow \{0, 1\}$, a classical (and often compact) representation of F is as a Boolean circuit with $\wedge$ and $\vee$ gates, where the circuit inputs are literals over V . Such a circuit can be visualized as a rooted directed-acyclic graph in which nodes with incoming edges, also called *internal nodes*, are labeled by $\vee$ and $\wedge$ operators, and nodes with no incoming edges, also called *leaves*, are labeled by literals over X . A circuit with the above properties is also said to be represent a function in Negation Normal Form (NNF) – such circuits will be the only ones of interest to us, unless stated otherwise. Figure 1 shows an example of such a circuit. As is the case in this figure, multiple leaves of a circuit may have the same literal label. A circuit is said to represent a conjunctive normal form (CNF) formula if it is of the form $\bigwedge_{i=1}^k C_i$, where each C_i is a disjunction of literals.

Suppose the set of variables V is partitioned into two parts: $X = \{x_1, \dots, x_m\}$ representing “outputs”, and $I = \{i_1, \dots, i_n\}$ representing “inputs”. A circuit C with leaves labeled by literals over $X \cup I$ represents a (Boolean function, and hence) relational specification over the inputs and outputs. For notational convenience, we use $C(X, I)$ to denote both a circuit with leaves labeled by literals over $X \cup I$, and the Boolean function over $X \cup I$ represented by such a circuit, depending on the context. Specifically, the terms Boolean function and Boolean circuit (or simply circuit) representing a Boolean function, are used interchangeably throughout the paper, although it will be clear from the context which of the two we mean. Given a relational specification $C(X, I)$, the *Boolean Functional Synthesis* problem asks us to find a set of Boolean functions $\Psi(I) = \{\psi_i(I) \mid 1 \leq i \leq |X|\}$ such that $\forall X (C(\Psi(I), I) \iff \exists X C(X, I))$. As we have seen above, this is an important problem with diverse applications. The function $\psi_i(I)$ is also called a *Skolem function* for x_i in $C(X, I)$, and $\Psi(I)$ is called a Skolem function vector for X . For example, if $X = \{a, b\}$ and $I = \{i\}$ in the circuit of Figure 1, then one (of possibly many) Skolem function vectors is $\Psi(i) = (\psi_a(i) = \neg i, \psi_b(i) = \neg i)$.

A set L of leaves of a circuit $C(X, I)$ is said to be *literal-consistent* if every leaf in L is labeled by the same literal. For a literal-consistent set of leaves L and for $b \in \{0, 1\}$, we use $C|_{L:b}$ to denote the circuit obtained by re-labeling each leaf in L with b . For a literal ℓ over $X \cup I$, we use the term ℓ -*leaves* of C to denote the set of all leaves of C labeled ℓ . For a set of distinct literals $\{\ell_1, \dots, \ell_i\}$ and (possibly same) labels $b_1, \dots, b_i$, we abuse notation and use $C|_{\ell_1=b_1, \dots, \ell_i=b_i}$ to denote the circuit obtained by re-labeling all ℓ_j -leaves of C by b_j , for all $j \in \{1, \dots, i\}$. Note that since ℓ and $\neg \ell$ are different literals, the notation $C|_{\ell=b, \neg \ell=b}$ is meaningful (and useful), and represents the circuit obtained by re-labeling all ℓ -leaves and $\neg \ell$ -leaves of C by b .

3. The Subset And-Unrealizable Normal Form (SAUNF)

We start with a basic definition of interest.

Definition 1. Given a circuit $C(X, I)$ and a literal ℓ over $X \cup I$, let v_ℓ be the underlying variable of ℓ . We say that ℓ is $\wedge$ -realizable in C iff there exists an assignment $\sigma : X \cup I \setminus \{v_\ell\} \rightarrow \{0, 1\}$ such that the Boolean function represented by $C|_{\ell=w, \neg \ell=w'}$ is semantically equivalent to $w \wedge w'$ under the assignment σ , where w, w' are fresh variables not in $X \cup I$. Further, we say that ℓ is $\wedge$ -unrealizable in C if it is not $\wedge$ -realizable.

Clearly, if ℓ is $\wedge$ -realizable (resp. $\wedge$ -unrealizable) in C , then so is $\neg \ell$. As an example, consider the circuit $C(a, b, i)$ in Figure 1, and let C_{left} and C_{right} denote the sub-circuits rooted at the left and right child, respectively of the root node. Then b is $\wedge$ -realizable in C_{right} and also in C , but is $\wedge$ -unrealizable in C_{left} .

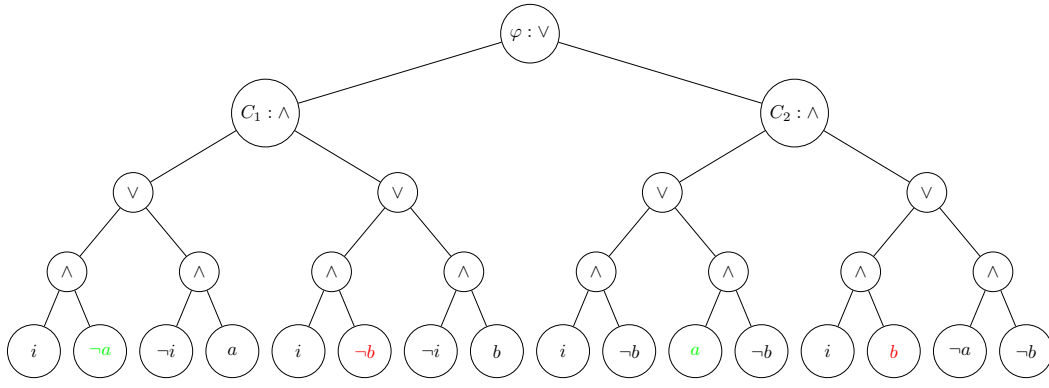


Figure 1: Example of an SAUNF circuit not in SynNNF. Input is $\{i\}$ and the output variables are $\{a, b\}$

We now extend the notion of $\wedge$ -(un)realizability to that of sets of literal-consistent leaves.

Definition 2. Let S be the set of all ℓ -leaves of C . A subset S' of S is said to be $\wedge$ -realizable (resp. $\wedge$ -unrealizable) in C if ℓ is $\wedge$ -realizable (resp. $\wedge$ -unrealizable) in $C|_{S \setminus S':0}$.

As an example, consider the set S' of leaves labeled a (in black) in Figure 1. This is a singleton set containing the 4th leaf from the left. The entire set S of a -leaves, however, consists of the 4th and 11th leaves from the left. In order to check $\wedge$ -(un)realizability of S' , we relabel the 11th leaf (labeled a in Figure 1) with 0, the 4th leaf (labeled a in black) with a new variable w , and all leaves labeled $\neg a$ with w' . It can now be verified that no assignment of b and i renders the function represented by the resulting circuit semantically equivalent to $w \wedge w'$. Hence S' is $\wedge$ -unrealizable in the circuit of Figure 1. We now use the above definitions to introduce a new normal form for Boolean circuits that precisely characterizes efficient Boolean functional synthesis.

Definition 3. A circuit $C(X, I)$ is said to be in the Subset And-Unrealizable Normal Form (SAUNF, for short) w.r.t. X and a sequence $S = (S_1, S_2, \dots, S_k)$ of subsets of leaves if the following hold:

1. For each $i \in \{1, \dots, k\}$, all leaves in S_i are labeled by the same literal over X .
2. S_1 is $\wedge$ -unrealizable in C .
3. For each $i \in \{2, \dots, k\}$, S_i is $\wedge$ -unrealizable in $C|_{S_1:1, S_2:1, \dots, S_{i-1}:1}$.
4. The function represented by $C|_{S_1:1, S_2:1, \dots, S_k:1}$ is semantically independent of X .

Consider the circuit $C(a, b, i)$ in Figure 1 with inputs $I = \{i\}$ and outputs $X = \{a, b\}$. Let the leaves of the circuit be denoted by $L_0, \dots, L_{15}$, where L_i denotes the $i + 1$ th leaf from the left in Figure 1. Now, consider the sequence S of subsets of leaves given by $S = (\{L_3\}, \{L_7\}, \{L_5\}, \{L_1\})$. This corresponds to subsets of leaves with the (coloured) labels $a, b, \neg b$ and $\neg a$. As seen above, $\{L_3\}$ is $\wedge$ -unrealizable in C . It can be verified that $\{L_7\}$ is $\wedge$ -unrealizable in $C|_{\{L_3\}:1}$,

$\{L_5\}$ is $\wedge$ -unrealizable in $C|_{\{L_3\}:1, \{L_7\}:1}$ and $\{L_1\}$ is $\wedge$ -unrealizable in $C|_{\{L_3\}:1, \{L_7\}:1, \{L_5\}:1}$. Finally, the function represented by $C|_{\{L_3\}:1, \{L_7\}:1, \{L_5\}:1, \{L_1\}:1}$ is semantically equivalent to 1, and hence independent of $\{a, b\}$. Hence, the circuit C is in SAUNF w.r.t. $X = \{a, b\}$ and the sequence $(\{L_3\}, \{L_7\}, \{L_5\}, \{L_1\})$.

Relation with other normal forms

A normal form for Boolean functional synthesis, called SynNNF, was recently proposed in (Akshay et al. 2019), where a conditional result was shown, namely, SynNNF is super-polynomially more succinct than DNNF (Darwiche 2001a) and dDNNF (Darwiche 2001b), unless some long-standing complexity theoretic conjectures are falsified. We use a result from (Bova et al. 2016) to show the following stronger result.

Lemma 1. SAUNF is unconditionally exponentially more succinct than DNNF and dDNNF.

For lack of space, we omit the proof of Lemma 1 here. All omitted proofs are presented in the supplementary material. Next, we show that SynNNF is in fact a very special case of SAUNF, by showing that every SynNNF circuit is a SAUNF circuit, but not vice versa.

Lemma 2. Every SynNNF circuit $C(X, I)$ is also an SAUNF circuit with a sequence of $2|X|$ $\wedge$ -unrealizable subsets of leaves. However, there exist SAUNF circuits that are not in SynNNF.

We remark that the circuit in Figure 1, already shown to be in SAUNF, is not in SynNNF. This is because the definition of SynNNF (Akshay et al. 2019) entails that for a circuit $C(X, I)$ to be in SynNNF, at least one output in X must be $\wedge$ -unrealizable in C . However, neither a nor b is $\wedge$ -unrealizable in the circuit. Specifically, $C|_{a=w, \neg a=w'} \Leftrightarrow w \wedge w'$ when $i = 0$ and $b = 0$, and $C|_{b=w, \neg b=w'} \Leftrightarrow w \wedge w'$ when $i = 1$ and $a = 1$.

4. SAUNF and Skolem functions

We now discuss how Boolean functional synthesis can be efficiently solved if the relational specification $C(X, I)$ is in SAUNF. Informally, the process involves

transforming C to a semantically different but related specification $D(X, X', I)$, where X' is a set of fresh output variables, such that a Skolem function vector for $X \cup X'$ in D can be found efficiently, and a projection of this Skolem function vector on X directly yields a Skolem function vector for X in C . To formalize this notion, we begin with a few definitions.

Definition 4. Equisynthesizable Under Projection. Let $C(X, I)$ and $D(X, X', I)$ be circuits representing relational specifications on inputs I and outputs X and $X \cup X'$, respectively. We say that C is equisynthesizable to D under projection, represented as $C \rightsquigarrow D$, iff the following hold

- $\forall I \forall X (C(X, I) \implies \exists X' D(X, X', I))$
- $\forall I \forall X \forall X' (D(X, X', I) \implies C(X, I))$

It follows from the above definition that $\rightsquigarrow$ is transitive. The following theorem is easily seen to hold.

Theorem 1. If $C(X, I) \rightsquigarrow D(X, X', I)$ and $|D|$ grows polynomially with $|C|$, then the following hold:

1. If Boolean function synthesis for $D(X, X', I)$ can be solved in time polynomial in $|D|$, the same problem for $C(X, Y)$ can be solved in time polynomial in $|C|$.
2. If there exists a Skolem function vector for $X \cup X'$ in D of size polynomial in $|D|$, there exists a Skolem function vector for X in C of size polynomial in $|C|$.

Role of auxiliary output variables

Definition 4 and Theorem 1 hint at the important role played by auxiliary output variables X' that are present in D , but not in C . We now investigate how these auxiliary variables can be introduced, and how they help in generating Skolem functions.

Let C_1 and C_2 be two sub-circuits of $C(X, I)$ such that C_1 is not a sub-circuit of C_2 and vice versa. For a fresh auxiliary variable $p \notin X \cup I$ and for $i \in \{1, 2\}$, define a circuit transformation τ_i^p that replaces the sub-circuit C_i in C with $C_i \wedge p$.

Lemma 3. If the Boolean function represented by $C_1 \wedge C_2$ is unsatisfiable, then $C \rightsquigarrow \tau_1^p(\tau_2^{-p}(C))$.

Proof. Let $D(X, p, I)$ denote the circuit $\tau_1^p(\tau_2^{-p}(C))$, and let D_1 and D_2 denote the sub-circuits $p \wedge C_1$ and $\neg p \wedge C_2$ respectively in D . We show below that the conditions for $C \rightsquigarrow D$ (see Definition 4) are satisfied.

Let $\sigma : X \cup I \rightarrow \{0, 1\}$ be an assignment for which $C(X, I)$ evaluates to 1. We have four cases to analyze depending on what C_1 and C_2 evaluate to under σ .

- $C_1 = 0 = C_2$, then for any value of p , D_1 and D_2 also evaluate to 0, and hence D evaluates to the same value, viz. 1, as C .
- If $C_1 = 1, C_2 = 0$, then with $p = 1$, D_1 evaluates to 1 and D_2 evaluates to 0, and hence D evaluates to the same value, viz. 1, as C .
- By a similar argument, if $C_1 = 0, C_2 = 1$, using $p = 0$ causes D to evaluate to 1.

- The case of $C_1 = C_2 = 1$ doesn't arise since $C_1 \wedge C_2$ always evaluates to 0.

This shows that $\forall I \forall X (C(X, I) \implies D(X, p, I))$.

Consider any assignment $\sigma' : X \cup \{p\} \cup I \rightarrow \{0, 1\}$ that causes D to evaluate to 1. Let $\sigma : X \cup I \rightarrow \{0, 1\}$ be the projection of σ' on $X \cup I$. By definition, the function represented by $D|_{p=1, \neg p=1}$ is semantically equivalent to C . Since all internal gates in D , viz. $\wedge$ and $\vee$ gates, are monotone, σ must cause $D|_{p=1, \neg p=1}$ to evaluate to 1 as well. Hence, σ causes $C(X, I)$ to evaluate to 1. This shows that $\forall I \forall X \forall p (D(X, p, I) \implies C(X, I))$. $\square$

The argument in the above proof can be easily generalized to prove the following.

Lemma 4. Let $\mathcal{C}_1 = \{C_{1,1}, \dots, C_{1,s}\}$ and $\mathcal{C}_2 = \{C_{2,1}, \dots, C_{2,t}\}$ be two sets of sub-circuits of $C(X, I)$ such that (a) there are no distinct $C_{k,i}$ and $C_{l,j}$ where one is a sub-circuit of the other, and (b) $\bigvee_{i=1}^s C_{1,i} \implies \bigwedge_{j=1}^t \neg C_{2,j}$, where we have used $C_{k,l}$ to denote the Boolean function represented by the corresponding circuit. Let $\tau_{\mathcal{C}_k}^p$ denote the circuit transformation that replaces every sub-circuit $C_{k,i}$ with $p \wedge C_{k,i}$. Then $C \rightsquigarrow \tau_{\mathcal{C}_1}^p(\tau_{\mathcal{C}_2}^{-p}(C))$.

A particularly easy application of Lemma 4 is obtained by choosing any literal ℓ that labels leaves of C , and by choosing $\mathcal{C}_1$ and $\mathcal{C}_2$ to be subsets of ℓ -leaves and $\neg \ell$ -leaves, respectively. Note that if L is a subset of ℓ -leaves of C , then $\tau_L^p(C)$ gives the same circuit as $C|_{L:p \wedge \ell}$.

Given a relational specification $C(X, I)$ and a literal ℓ over X , let v_ℓ be the underlying variable in X , and S_ℓ be the set of all ℓ -leaves of $C(X, I)$.

Theorem 2. Suppose $S \subseteq S_\ell$ is $\wedge$ -unrealizable in $C(X, I)$. For a fresh auxiliary output variable $p \notin X \cup I$, let $E(X, p, I) = C|_{S_\ell \setminus S: p \wedge \ell, S_\ell \setminus S: \neg p \wedge \neg \ell}$ and $D(X \setminus \{v_\ell\}, p, I) = E|_{\ell=1, \neg \ell=1}$. Then the following statements hold.

1. $\exists v_\ell C \iff \exists p D \iff \exists v_\ell C|_{S:1}$
2. If $\Psi_D(I)$ is a Skolem function vector for $X \setminus \{v_\ell\} \cup \{p\}$ in D , then the projection of Ψ_D on $X \setminus \{v_\ell\}$ augmented with the function $E|_{\ell=1, \neg \ell=0}(\Psi_D(I), I)$ for ℓ gives a Skolem function vector $\Psi_C(I)$ for X in C .

Proof. By Lemma 4, we have $C \rightsquigarrow E$. It then follows from Definition 4 that $\exists v_\ell C \iff \exists v_\ell \exists p E$. Furthermore, since S is $\wedge$ -unrealizable in C , it follows from the definition of E and Definition 2 that v_ℓ is $\wedge$ -unrealizable in E . Therefore, by a result of (Akshay et al. 2018), we have $\exists v_\ell E \iff E|_{\ell=1, \neg \ell=1} \iff D$. Hence, $\exists v_\ell \exists p E \iff \exists p D$ (as shown in (Akshay et al. 2018)).

Since $\exists v_\ell E \iff D$, the Skolem function vector for $X \setminus \{v_\ell\}$ in E is $\Psi_D(I)$ projected on $X \setminus \{v_\ell\}$. The Skolem function vector for ℓ is given by $E|_{\ell=1, \neg \ell=0}(\Psi_D(Y), 1, Y)$, as described in (Akshay et al. 2018; Jiang 2009; Fried, Tabajara, and Vardi 2016;

Algorithm 1: SkGen(C, S, r)

Input: $C(X, Y)$: Relational spec in SAUNF;
 $S = (S_1, S_2, \dots, S_k)$: Sequence of $\wedge$ -unrealizable subsets of X -leaves of C ; r : Recursion level
Output: $\Psi_C(Y)$: Skolem function vector for C

```

1 if  $r = k+1$  then
2    $\Psi_C(Y) := \text{GetAnyFuncVec}(|X|, Y);$ 
   // Returns an  $|X|$ -dim vector of
   (arbitrary) functions of  $Y$ 
3 else
4    $\ell := \text{Literal label of leaves in } S_r;$ 
5    $p_r := \text{newOutputVar}();$ 
6    $E(X', \ell, Y) := \text{GetCkt}(C, S_r, \ell, p_r);$ 
   // Replace all  $\ell$ -labeled leaves of  $C$ 
   other than those in  $S_r$  by  $\ell \wedge p_r$ ,
   and replace all  $\neg\ell$ -labeled leaves
   by  $\neg\ell \wedge \neg p_r$ 
7    $S := \text{GetNewSeq}(S, r, \ell, p_r);$ 
   // Replace  $\ell$  by  $p_r$  and  $\neg\ell$  by  $\neg p_r$  in
   all leaves in  $S_j$  for  $j > r$ 
8    $D := \text{CPropSimp}(E \mid_{\ell=1, \neg\ell=1});$ 
   // CPropSimp propagates constants and
   eliminates gates with constant
   outputs
9    $\Psi_D(Y) = \text{SkGen}(D, S, r+1);$ 
10   $\psi_\ell(Y) := \text{CPropSimp}(E(\Psi_D(Y), \ell = 1, Y));$ 
11   $\Psi_C(Y) = (\Psi_D(Y) \setminus \psi_D^{p_r}(Y), \psi_\ell(Y))$ 
12 return  $\Psi_C(Y);$ 

```

	$C^{(r)}$	$E^{(r)}$	$D^{(r)}$
$r=1$:	None	$L_{\{1,14\}}:(\neg a \wedge \neg p_1),$ $L_{\{10\}}:(a \wedge p_1)$	$L_{\{1,14\}}:\neg p_1, L_{\{3\}}:1,$ $L_{\{10\}}:p_1$
$r=2$:	Same as in $D^{(1)}$	$L_{\{1,14\}}:\neg p_1,$ $L_{\{3\}}:1, L_{\{10\}}:p_1,$ $L_{\{5,9,11,15\}}:\neg b \wedge \neg p_2,$ $L_{\{13\}}:b \wedge p_2$	$L_{\{1,14\}}:\neg p_1,$ $L_{\{3,7\}}:1, L_{\{10\}}:p_1,$ $L_{\{5,9,11,15\}}:\neg p_2,$ $L_{\{13\}}:p_2$
$r=3$:	Same as in $D^{(2)}$	$L_{\{1,14\}}:\neg p_1,$ $L_{\{3,7\}}:1, L_{\{10\}}:p_1,$ $L_{\{9,11,15\}}:\neg p_2 \wedge p_3,$ $L_{\{5\}}:\neg p_2,$ $L_{\{13\}}:p_2 \wedge \neg p_3$	$L_{\{1,14\}}:\neg p_1,$ $L_{\{3,5,7\}}:1, L_{\{10\}}:p_1,$ $L_{\{9,11,15\}}:p_3,$ $L_{\{13\}}:\neg p_3$
$r=4$:	Same as in $D^{(3)}$	$L_{\{1\}}:\neg p_1,$ $L_{\{14\}}:\neg p_1 \wedge p_4,$ $L_{\{3,5,7\}}:1,$ $L_{\{10\}}:p_1 \wedge \neg p_4,$ $L_{\{9,11,15\}}:p_3,$ $L_{\{13\}}:\neg p_3$	$L_{\{14\}}:p_4,$ $L_{\{1,3,5,7\}}:1,$ $L_{\{10\}}:\neg p_4,$ $L_{\{9,11,15\}}:p_3,$ $L_{\{13\}}:\neg p_3$

Table 1: Run of Algorithm 4 on Fig. 1

Each entry in this table lists which leaf labels of C must be updated, where $L_{\{i,j,k\}}:f$ denotes updation of the label of each leaf in $\{L_i, L_j, L_k\}$ by f . All leaves whose label updates are not specified are assumed to have the same labels as in C . It can be verified that C with leaf labels updated as in the table entry corresponding to $D^{(4)}$ simplifies to 1 by constant propagation. Hence $D^{(4)}$ is semantically independent of output variables. This is not a coincidence, but is guaranteed by the definition of SAUNF. Hence, at recursion level 5 of *SkGen*, any vector of functions $(f_3(i), f_4(i))$ can be returned in line 2 of Algorithm 4 as a Skolem function vector for (p_3, p_4) in $D^{(2)}$.

As the recursive calls return, we obtain $E^{(4)}(p_3 = f_3(i), p_4 = f_4(i), p_1 = 1, i)$ as Skolem function for p_1 in $E^{(4)}$. Call this function $f_1(i)$. Next, we get $f_2(i) = E^{(3)}(p_1 = f_1(i), p_3 = f_3(i), p_2 = 1, i)$ as Skolem function for p_2 in $E^{(3)}$. Continuing further, we obtain $f_b(i) = E^{(2)}(p_1 = f_1(i), p_2 = f_2(i), b = 1, i)$ as Skolem function for b in $E^{(2)}$, and $f_a(i) = E^{(1)}(b = f_b(i), p_1 = f_1(i), a = 1, i)$ as Skolem function for a in $E^{(1)}$. The final return gives $(f_a(i), f_b(i))$ as a Skolem function vector for (a, b) in C . Note that different choices of $f_3(i), f_4(i)$ yield different Skolem function vectors of C , all of which are correct.

Theorem 3. *Given a specification $C(X, I)$ in SAUNF Algorithm SkGen generates a polynomial-sized Skolem function vector in polynomial-time.*

Next, we show that if we already know one (out of possibly many) Skolem function vector of a relational specification $C(X, I)$, we can easily derive a semantically equivalent specification in SAUNF.

Lemma 5. *Let $\Psi(I)$ be a Skolem function vector for X in $C(X, I)$. Define $G(I)$ to be $C(\Psi(I), I)$, $\tilde{F}(X, I)$ to be $\bigwedge_{i=1}^n ((x_i \wedge \psi_i) \vee (\neg x_i \wedge \neg \psi_i))$, and $H(X, I)$ to be $(\tilde{F}(X, I) \vee F(X, I)) \wedge G(I)$. Then $H(X, I)$ is in SAUNF*

Trivedi 2003). The second part of the theorem now follows from the observation that $C \rightsquigarrow E$. $\square$

Theorem 2 suggests an efficient algorithm for generating a Skolem function vector from a SAUNF specification. Algorithm 4 presents the pseudo-code of algorithm *SkGen*. The purpose of sub-routines used in *SkGen* is explained in comments. If $|C|$ denotes the size (i.e. count of nodes and edges) of the circuit C , it is easy to see that *SkGen* runs in $\mathcal{O}(k^2 \cdot |C|)$ time, assuming the vector returned in line 2 can be constructed in time $\mathcal{O}(k^2 \cdot |C|)$. Since any arbitrary function vector suffices in line 2, the assumption is easily satisfied by choosing constant functions, for example.

We illustrate the running of *SkGen* by considering its execution on the specification $C(a, b, i)$ shown in Fig. 1. Here, the outputs are $X = \{a, b\}$ and the input is $I = \{i\}$. As discussed earlier, we use L_0 through L_{15} to denote the leaves of the circuit C in left-to-right order. We have also seen earlier that C is in SAUNF for the sequence of subsets of leaves (S_1, S_2, S_3, S_4) , where $S_1 = \{L_3\}$, $S_2 = \{L_7\}$, $S_3 = \{L_1\}$ and $S_4 = \{L_5\}$.

As algorithm *SkGen* proceeds, labels of different leaves of C need to be updated. For notational convenience, we use $C^{(r)}$, $D^{(r)}$ and $E^{(r)}$ to refer to the circuits C , D and E in the r^{th} level of recursion of *SkGen*. Table 1 shows how $C^{(r)}$, $D^{(r)}$ and $E^{(r)}$ are obtained by replacing the labels of suitable leaves of C .

and both $H(X, I)$ and $C(X, I)$ represent semantically equivalent specifications.

In the above Lemma, $H(X, I)$ is the required specification. The following theorem is an immediate consequence of Lemma 5 and Theorem 3.

Theorem 4. *Given a relational specification $C(X, I)$,*

1. *A Skolem function vector can be computed in polynomial time iff a semantically equivalent SAUNF form can be computed in polynomial time.*
2. *A Skolem function vector has polynomial size iff there exists a semantically equivalent SAUNF form of the specification that is polynomial sized.*

Thus, SAUNF truly characterizes efficient Boolean functional synthesis. Note that Theorem 4 is significantly stronger than sufficient conditions for efficient synthesis given in (Akshay et al. 2018; 2019).

5. Operations on SAUNF

In this section, we discuss the application of basic operations like conjunction, disjunction and existential quantification of variables on SAUNF representations.

Lemma 6. *Suppose $G(X, I)$ is in SAUNF w.r.t X and a sequence S^G , and $H(X, I)$ is in SAUNF w.r.t. X and S^H . Then $G \vee H$ obtained by adding a two-input $\vee$ gate fed by outputs of G and H is in SAUNF w.r.t X and (S^G, S^H) .*

Proof. The core of the proof is the claim that (S^G, S^H) is a sequence of subsets of leaves of $G \vee H$ that satisfies the conditions of Definition 3. To see why this is so, note that by Definition 2, when considering a subset, say S_i^G , of ℓ -labeled leaves in S^G , all ℓ -labeled leaves of H must be re-labeled 0. Hence, H can only contribute $\neg\ell$, and can, at worst, combine with ℓ contributed by G at the $\vee$ -labeled root of $G \vee H$. Since the set of ℓ -labeled leaves in S_i^G was already $\wedge$ -unrealizable in G , we find that S_i^G is $\wedge$ -unrealizable in $G \vee H$ as well. By repeating this argument, we find that when all subsets of leaves in (S^G, S^H) are set to 1, the circuit $G \vee H$ represents a function that is semantically independent of X . Hence, $G \vee H$ is in SAUNF. $\square$

Significantly, Lemma 6 does not require any assumptions on the relation between ordering of subsets in S^G and S^H . Other popular normal forms do not enjoy this property. For example, disjoining two ROBDDs (Reduced Ordered Binary Decision Diagrams) constructed with different ordering of variables does not always yield an ROBDD in polynomial-time. Similarly, combining two SynNNF circuits with an $\vee$ gate may not result in a SynNNF circuit unless the ordering of output variables in both circuits are the same. This shows that disjunction is more efficiently computable in SAUNF than in ROBDDs and in SynNNF.

Lemma 7. *Suppose $C(X, I)$ in SAUNF w.r.t X and a sequence S . Let L be the set of all leaves of C that are labeled by a literal over X . Then $\exists X C \Leftrightarrow C|_{L:1}$.*

Proof. Follows from Theorem 2(1) and Definition 3. $\square$

Lemma 8. *Suppose $G(X, I)$ is in SAUNF w.r.t X and a sequence S^G , and $H(X, I)$ is in SAUNF w.r.t. X and S^H . If there is no literal ℓ over X such that G has an ℓ -labeled leaf and H has a $\neg\ell$ -labeled leaf, combining the outputs of G and H by an $\wedge$ -gate gives a circuit in SAUNF w.r.t. X and (S^G, S^H) . Otherwise, an SAUNF circuit semantically equivalent to $G \wedge H$ cannot be constructed in time polynomial in $|G|, |H|$ unless $P = NP$. Such a circuit cannot have size polynomial in $|G|, |H|$ unless $\Pi_2^P = \Sigma_2^P$ (i.e. the polynomial hierarchy collapses to the second level).*

Finally, we ask how difficult it is to check if a given circuit $C(X, I)$ is in SAUNF.

Theorem 5. 1. *Given $C(X, I)$ and a sequence of literal-consistent disjoint subsets of leaves $S = (S_1, \dots, S_k)$, checking if C is in SAUNF w.r.t. X and S is co-NP complete.*

2. *Given $C(X, I)$, checking if C is in SAUNF w.r.t. X and some (unspecified) sequence of subsets of leaves is co-NP hard and in Σ_2^P .*

6. Conversion to SAUNF

We now discuss an algorithm for transforming a circuit $C(X, I)$ representing a CNF formula to a semantically equivalent circuit in SAUNF. Algorithm 3 uses a routine "GetSubset" for obtaining an $\wedge$ -unrealizable subset of leaves for a chosen literal ℓ . The pseudocode for "GetSubset" is shown in Algorithm 2. Let v_ℓ denote the underlying variable of the literal ℓ . In Algorithm 2, $CurrS$ is a subset of clauses containing ℓ such that there exists an assignment $\sigma : X \setminus \{v_\ell\} \cup I \rightarrow \{0, 1\}$ for which all (and only) these clauses of C evaluate to ℓ , and ℓ is $\wedge$ -realizable in C under σ (see Definition 1). To ensure that $CurrS$ in the current iteration does not include any such sets obtained in previous iterations of the loop, we conjoin $\text{DisjoinWithoutLit}(CurrS, \ell)$. This ensures that $CurrS$ generated in any iteration skips at least one clause from any previously generated $CurrS$. All sets $CurrS$ obtained as the repeat-until loop iterates are collected in $AllS$. Finally when ℓ becomes $\wedge$ -unrealizable in D , we obtain a (minimal) hitting set (set cover) $HitS$ of $AllS$, and exclude all ℓ -leaves that appear in the clauses in $HitS$ to obtain a (maximal) $\wedge$ -unrealizable subset of ℓ -leaves.

We now show that circuit returned by Algorithm 3 is in SAUNF. Note that C_1, C_2 are independent of ℓ . Therefore, in the circuit F constructed in line 11 of Algorithm 3, the set containing only the leaf ℓ (call it L_1) conjoined with C_1 is $\wedge$ -unrealizable as it meets up $\neg\ell$ at an $\vee$ gate. In $F|_{L_1:1}$, we can show that set $S^{C'}$ of ℓ -leaves of C' is $\wedge$ -unrealizable as it was already $\wedge$ -unrealizable in the circuit C . Therefore, in $F_{L_1:1, S^{C':1}}$, all positive leaves of ℓ have been set to 1, and therefore we can set the only $\neg\ell$ (L') to 1. $F_{L_1:1, S^{C':1}, L':1} = C_1 \vee C_2$, which is in SAUNF assuming the recursive calls return correct representations and

Algorithm 2: GetSubset(C, ℓ)

Input: C : CNF circuit, ℓ : Literal**Output:** T : $\wedge$ -unrealizable subset of ℓ -leaves

```
1  $AllS := CurrS := \emptyset$ ;  
2  $D := C$ ;  
3 repeat  
4    $\sigma := \text{GetAssignment}(D, \ell)$ ;  
   // Assignment of vars other than  $v_\ell$   
   // for which  $\ell$  is  $\wedge$ -realizable in  $D$   
5    $CurrS := \text{GetClausesEvaluatingToL}(C, \sigma, \ell)$ ;  
   // Set of all clauses of  $C$  containing  
   //  $\ell$  that do not become 1 under  $\sigma$   
6    $AllS := AllS \cup \{CurrS\}$ ;  
7    $D := D \wedge \text{DisjoinWithoutLit}(CurrS, \ell)$ ;  
   // DisjoinWithoutLit( $CurrS, \ell$ ) gives  
   // disjunction of clauses in  $CurrS$   
   // after dropping  $\ell$   
8 until  $\ell$  is  $\wedge$ -unrealizable in  $D$ ;  
9  $HitS := \text{HittingSet}(AllS)$ ;  
10  $T := \text{Set of all } \ell\text{-leaves of } C \text{ that don't feed into}$   
   // any clause in  $HitS$ ;  
11 return  $T$ ;
```

using Lemma 6. The base condition is when C is independent of X , for which it already returns the correct value. This algorithm represents a significant reduction in size over Shannon expansion, as we expand the circuit D obtained after setting certain clauses of C to 1 (line 6 of Algorithm 3). In the worst-case, however, this reduces to a Shannon-expansion based conversion of C .

Algorithm 3: GetSAUNF(C, I)

Input: $C(X, I)$: Ckt representing CNF formula**Output:** $F(X, I)$: SAUNF ckt sem. equiv. to C

```
1 if  $C$  is semantically independent of  $X$  then  
2   return  $C$ ;  
3  $\ell := \text{ChooseLiteral}(C)$ ;  
   // Gives a literal over  $X$  in  $C$   
4  $S := \text{GetSubset}(C, \ell)$ ;  
5  $C' := \text{CktFromClausesWithLeaves}(S)$ ;  
   //  $C'$  represents conjunction of clauses  
   // of  $C$  containing leaves in  $S$   
6  $D := C|_{S=1}$ ;  
7 if  $D$  is semantically independent of  $\ell$  then  
8   return  $C' \wedge \text{GetSAUNF}(D)$ ;  
9  $C_1 := \text{GetSAUNF}(D|_{\ell=1})$ ;  
10  $C_2 := \text{GetSAUNF}(D|_{\ell=0})$ ;  
11  $F := C' \wedge ((\ell \wedge C_1) \vee (\neg \ell \wedge C_2))$ ;  
12 return  $F$ ;
```

7. Some interesting applications

In Section 1, we had discussed applications of Boolean functional synthesis to synthesis of real-time monitors

in safety-critical autonomous systems and to inversion of binarized neural networks. In this section, we discuss some other interesting applications. Specifically, we consider the *bounded integer factorization problem* – a problem of immense interest in cryptanalysis, and show some interesting partial results using SAUNF circuits. The relational specification C_{fact} for factorization is given by $X \times Y = I$ which outputs 1 if and only if the given product relation holds, where X, Y are n -bit outputs vectors and I is a $2n$ bit input vector. A bit at position j refers to the j^{th} least significant bit of the vector. For $1 \leq i \leq j \leq 2n$, define $R[i, j]$ to be the relation that outputs 1 if and only if the bits from position i to j of $X \times Y$ are equal to bits from position i to j of I .

From the results presented in this paper, it follows that if $R[1, 2n] \wedge (X \neq 1) \wedge (Y \neq 1)$ can be represented as a SAUNF circuit of size $\alpha(n)$, (Skoem function) circuits of size polynomial in $\alpha(n)$ can be obtained for bounded integer factorization with non-trivial (i.e. $\neq 1$) factors. This would have serious ramifications for cryptanalysis. While we are not close to achieving such a result, our initial studies show some interesting results in trying to represent $R[i, j]$ in SAUNF. Note that it is already known (Bryant 1991), that representing $R[n, n]$ requires exponentially large ROBDDs. With SAUNF circuits however, we obtain a significant improvement.

Theorem 6. For $i \leq j \leq 2n, j - i < n, R[i, j]$ is representable by a polynomial (in n) sized SAUNF circuit.

To the best of our knowledge, such a result is not known for other normal forms like DNNF, dDNNF, or even SynNNF. Surprisingly, we can use Theorem 6 to also show that a restricted version of division has a polynomial sized SAUNF representation. Consider n -bit numbers X, Y and a $2n$ -bit number I such that the relational specification circuit outputs 1 iff $X \times Y = I$ holds. For the division problem, we treat I, Y as inputs to the problem and X as the output and write the relation as $X = I/Y$.

Theorem 7. The relation $X = I/Y$, with inputs I, Y restricted to odd numbers (i.e. the relation evaluates to 0 if I or Y is even), is representable as a polynomial (in n) sized SAUNF circuit.

8. Conclusion

In this paper, we presented a normal form SAUNF for relational specifications that characterizes efficient Boolean functional synthesis. This is a significantly stronger characterization than those used in earlier work. SAUNF is (unconditionally) exponentially more succinct than DNNF, dDNNF while enjoying similar composability properties. It also strictly subsumes the recently proposed SynNNF. As future work, we plan to implement the algorithm to transform a given circuit to SAUNF and apply it on benchmarks from cryptanalysis, ML-based controller synthesis and inverting binarized neural nets.

Broader Impact and Ethics Statement

Identifying and exploiting knowledge representations for various problems (e.g., model counting, neural net analysis etc) has been a classical area of study in AI. For example, knowledge representations for model counting has been considered by researchers for more than two decades and this has led a deeper understanding of this problem and its potential applications.

In this work, we have focussed on the specific problem of Boolean functional synthesis and undertaken a study of knowledge representation characterization for this particular problem. We have also pointed out two/three potential applications of this problem in an AI and cryptanalysis settings. Understanding the structure of the specification in these domains via the lens of knowledge representation allows us better insight into the problems and hopefully to build better algorithms with theoretical and rigid guarantees. This is indeed one of the holy grails of AI and the current paper could be seen as a step in the direction.

We understand however that the techniques in the paper can potentially lead to reverse engineering (binarized) neural networks and other systems representable as Boolean Functions. Thus one could obtain functional insight into opaque black-box systems (like cryptosystems). We hope that with sound regulatory mechanisms, potential abuse can be avoided. We also hope that this work at its foundational level would help researchers in these domains understand and hence build techniques to counter and avoid such approaches.

References

- Akshay, S.; Chakraborty, S.; John, A. K.; and Shah, S. 2017. Towards Parallel Boolean Functional Synthesis. In *TACAS 2017 Proceedings, Part I*, 337–353.
- Akshay, S.; Chakraborty, S.; Goel, S.; Kulal, S.; and Shah, S. 2018. What’s Hard About Boolean Functional Synthesis? In *Computer Aided Verification - 30th International Conference, CAV 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 14-17, 2018, Proceedings, Part I*, 251–269.
- Akshay, S.; Arora, J.; Chakraborty, S.; Krishna, S. N.; Raghunathan, D.; and Shah, S. 2019. Knowledge compilation for boolean functional synthesis. In *FMCAD*, 161–169. IEEE.
- Bova, S.; Capelli, F.; Mengel, S.; and Slivovsky, F. 2016. Knowledge compilation meets communication complexity. In Kambhampati, S., ed., *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, July 9-15 2016*, 1008–1014. IJCAI / AAAI Press.
- Bryant, R. E. 1991. On the complexity of vlsi implementations and graph representations of boolean functions with application to integer multiplication. *IEEE Transactions on Computers* 40(2):205–213.
- Cadoli, M., and Donini, F. M. 1997. A survey on knowledge compilation. *AI Commun.* 10(3-4):137–150.
- Chakraborty, S.; Fried, D.; Tabajara, L. M.; and Vardi, M. Y. 2018. Functional synthesis via input-output separation. In *2018 Formal Methods in Computer Aided Design, FMCAD 2018, Austin, TX, USA, October 30 - November 2, 2018*, 1–9.
- Darwiche, A., and Marquis, P. 2011. A knowledge compilation map. *CoRR* abs/1106.1819.
- Darwiche, A. 2001a. Decomposable negation normal form. *J. ACM* 48(4):608–647.
- Darwiche, A. 2001b. On the tractable counting of theory models and its application to truth maintenance and belief revision. *Journal of Applied Non-Classical Logics* 11(1-2):11–34.
- Fried, D.; Tabajara, L. M.; and Vardi, M. Y. 2016. BDD-based boolean functional synthesis. In *Computer Aided Verification - 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part II*, 402–421.
- Golia, P.; Roy, S.; and Meel, K. 2020. *Manthan: A Data-Driven Approach for Boolean Function Synthesis*. 611–633.
- Jiang, J.-H. R., and Balabanov, V. 2011. Resolution proofs and Skolem functions in QBF evaluation and applications. In *Proc. of CAV*, 149–164. Springer.
- Jiang, J. R. 2009. Quantifier elimination via functional composition. In *CAV*, volume 5643 of *Lecture Notes in Computer Science*, 383–397. Springer.
- John, A.; Shah, S.; Chakraborty, S.; Trivedi, A.; and Akshay, S. 2015. Skolem functions for factored formulas. In *FMCAD*, 73–80.
- Kuncak, V.; Mayer, M.; Piskac, R.; and Suter, P. 2010. Complete functional synthesis. *SIGPLAN Not.* 45(6):316–329.
- Lang, J.; Liberatore, P.; and Marquis, P. 2011. Propositional independence - formula-variable independence and forgetting. *CoRR* abs/1106.4578.
- Marijn Heule, M. S., and Biere, A. 2014. Efficient Extraction of Skolem Functions from QRAT Proofs. In *Formal Methods in Computer-Aided Design, FMCAD 2014, Lausanne, Switzerland, October 21-24, 2014*, 107–114.
- Muise, C.; McIlraith, S. A.; Beck, J. C.; and Hsu, E. 2016. DSHARP: Fast d-DNNF Compilation with sharpSAT. In *AAAI-16 Workshop on Beyond NP*.
- Rabe, M. N., and Seshia, S. A. 2016. Incremental determinization. In *Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings*, 375–392.
- Rabe, M. N., and Tentrup, L. 2015. CAQE: A certifying QBF solver. In *Formal Methods in Computer-Aided Design, FMCAD 2015, Austin, Texas, USA, September 27-30, 2015.*, 136–143.
- Trivedi, A. 2003. Techniques in symbolic model checking. Master’s thesis, Indian Institute of Technology Bombay, Mumbai, India.

Characterizing Boolean Functional Synthesis via Knowledge Representation (Supplementary Material)

Typos: Please consider the following small, but important, fixes of typos in the paper. The characters in **red** should be replaced by those in **blue**.

- Line 298: $C(X, \textcolor{red}{Y})$ should be $C(X, \textcolor{blue}{I})$
- Line 382: $E \mid_{\ell=1, \neg \ell=0} (\Psi_D(\textcolor{red}{Y}), 1, \textcolor{red}{Y})$ should be $E \mid_{\ell=1, \neg \ell=0} (\Psi_D(\textcolor{blue}{I}), 1, \textcolor{blue}{I})$.
- In Algorithm 1, all instances of **Y** should be replaced by **I** to be notationally consistent with the previous discussion.
- Line 446: $(\tilde{F}(X, I) \vee \textcolor{red}{F}(X, I)) \wedge G(I)$ should be $(\tilde{F}(X, I) \vee \textcolor{blue}{C}(X, I)) \wedge G(I)$.

Skipped proofs of Lemmas and Theorems:

Lemma 1. *SAUNF is unconditionally exponentially more succinct than DNNF and dDNNF.*

Proof. In Proposition 11 of (Bova et al. 2016), a family of Boolean functions JS_n is described that can be represented in CNF in size $\mathcal{O}(n^2)$, whereas any DNNF (and hence also dDNNF) representation of the function requires size $2^{\Omega(n^2)}$. The function JS_n asserts that for every triple (v_i, v_j, v_k) of variables in a carefully constructed set A_n of triples, at least one of v_i, v_j or v_k must be false. Furthermore, it is shown in (Bova et al. 2016) than $|A_n| = \mathcal{O}(n^2)$. Clearly, the CNF representation of JS_n has $|A_n|$ clauses, where each clause has three negated variables as literals. Therefore, no literal-consistent subset of leaves can be $\wedge$ -realizable in the circuit corresponding to this CNF representation. This implies that JS_n can be represented in SAUNF in size $\mathcal{O}(n^2)$. This establishes that SAUNF is unconditionally exponentially more succinct compared to DNNF and dDNNF. $\square$

To prove Lemma 2, we need to recall the definition of SynNNF from (Akshay et al. 2019). Using our notation, this definition can be written as follows.

Definition 5. *A circuit $C(X, I)$ is in SynNNF w.r.t. X and a linear ordering $x_1 \prec x_2 \dots \prec x_n$ of variables in X iff the following conditions hold:*

- x_1 is not $\wedge$ -realizable in C .
- For $2 \leq i \leq n$, x_i is not $\wedge$ -realizable in $C \mid_{x_1=1, \neg x_1=1, \dots, x_{i-1}=1, \neg x_{i-1}=1}$.

We can now prove the following lemma.

Lemma 2. *Every SynNNF circuit $C(X, I)$ is also an SAUNF circuit with a sequence of $2|X|$ $\wedge$ -unrealizable subsets of leaves. However, there exist SAUNF circuits that are not in SynNNF.*

Proof. This is a direct application of the definitions. Suppose a circuit $C(X, I)$ is in SynNNF, and let $|X| = n$. We use the ordering $x_1 \prec \dots \prec x_n$ of variables in Definition 5 to define a sequence of $2n$ literal-consistent subsets of leaves of C as follows. For each $i \in \{1, \dots, n\}$, we define S_{2i-1} to be the set of all leaves of C labeled x_i , and S_{2i} to be the set of all leaves of C labeled $\neg x_i$. It is now easy to see from Definition 5 and Definition 3 that C is in SAUNF w.r.t. X and the sequence $S = (S_1, \dots, S_{2n})$ of subsets of literal-consistent leaves. This proves the first part of the lemma.

To show the second part, we need to show a circuit that is in SynNNF but not in SAUNF. We have already discussed earlier that the circuit in Figure 1 satisfies this requirement. This completes the proof. $\square$

Theorem 1. *If $C(X, I) \rightsquigarrow D(X, X', I)$ and $|D|$ grows polynomially with $|C|$, then the following hold:*

1. *If Boolean function synthesis for $D(X, X', I)$ can be solved in time polynomial in $|D|$, the same problem for $C(X, I)$ can be solved in time polynomial in $|C|$.*
2. *If there exists a Skolem function vector for $X \cup X'$ in D of size polynomial in $|D|$, there exists a Skolem function vector for X in C of size polynomial in $|C|$.*

Proof. Since C and D are equisatisfiable under projection, for any assignment of X, X' and I that satisfies D , the projection of this assignment on $X \cup I$ must satisfy C . The Skolem function vector of C is then simply obtained from the Skolem function vector of D by dropping the Skolem functions of variables X' not present in C . Therefore, if a Skolem function vector for $|D|$ is computable in time (resp. has size) polynomial in $|D|$, we can obtain a Skolem function vector for $|C|$ in time (resp. of size) polynomial in $|D|$. Since $|D|$ grows polynomially with $|C|$, we have the result. $\square$

Lemma 4. *Let $\mathcal{C}_1 = \{C_{1,1}, \dots, C_{1,s}\}$ and $\mathcal{C}_2 = \{C_{2,1}, \dots, C_{2,t}\}$ be two sets of sub-circuits of $C(X, I)$ such that (a) there are no distinct $C_{k,i}$ and $C_{l,j}$ where one is a sub-circuit of the other, and (b) $\bigvee_{i=1}^s C_{1,i} \implies \bigwedge_{j=1}^t \neg C_{2,j}$, where we have used $C_{k,l}$ to denote the Boolean function represented by the corresponding circuit. Let $\tau_{\mathcal{C}_k}^p$ denote the circuit transformation that replaces every sub-circuit $C_{k,i}$ with $p \wedge C_{k,i}$. Then $C \rightsquigarrow \tau_{\mathcal{C}_1}^p(\tau_{\mathcal{C}_2}^{-p}(C))$.*

Proof. Let $D(X, p, I)$ denote the circuit $\tau_{\mathcal{C}_1}^p(\tau_{\mathcal{C}_2}^{-p}(C))$, and Let $\mathcal{D}_1 = \{p \wedge C_{1,1}, \dots, p \wedge C_{1,s}\}$ and $\mathcal{D}_2 = \{\neg p \wedge C_{2,1}, \dots, \neg p \wedge C_{2,t}\}$. We show below that the conditions for $C \rightsquigarrow D$ (see Definition 4) are satisfied.

Let $\sigma : X \cup I \rightarrow \{0, 1\}$ be an assignment for which $C(X, I)$ evaluates to 1. We have four cases to analyze depending on what $\bigvee_{i=1}^s C_{1,i}$ and $\bigwedge_{j=1}^t \neg C_{2,j}$ evaluate to under σ .

- $\bigvee_{i=1}^s C_{1,i} = 0$ and $\bigwedge_{j=1}^t \neg C_{2,j} = 0$: Clearly, all $C_{1,i} = 0$. Therefore, with $p = 0$, each sub-circuit in $\mathcal{D}_1$ and $\mathcal{D}_2$ evaluates to the same value as the corresponding

sub-circuit in $\mathcal{C}_1$ (each sub-circuit in $\mathcal{C}_1$ evaluates to 0) and $\mathcal{C}_2$ respectively. Hence D evaluates to the same value as C .

- $\bigvee_{i=1}^s C_{1,i} = 1$ and $\bigwedge_{j=1}^t \neg C_{2,j} = 1$: With $p = 1$, each sub-circuit in $\mathcal{D}_1$ and $\mathcal{D}_2$ evaluates to the same value as the corresponding sub-circuit in $\mathcal{C}_1$ and $\mathcal{C}_2$ (each sub-circuit in $\mathcal{C}_2$ evaluates to 0) respectively. Hence D evaluates to the same value as C .
- $\bigvee_{i=1}^s C_{1,i} = 0$ and $\bigwedge_{j=1}^t \neg C_{2,j} = 1$: Clearly, all $C_{1,i} = 0$ and all $C_{2,j} = 0$ as well. Therefore, for any value of p , each sub-circuit in $\mathcal{D}_1$ and $\mathcal{D}_2$ evaluates to the same value as the corresponding sub-circuit in $\mathcal{C}_1$ and $\mathcal{C}_2$ respectively (all subcircuits in both $\mathcal{C}_1$ and $\mathcal{C}_2$ evaluate to 0). Hence D evaluates to the same value as C .
- The only remaining case is when $\bigvee_{i=1}^s C_{1,i} = 1$ and $\bigwedge_{j=1}^t \neg C_{2,j} = 0$. This cannot happen because of the condition $\bigvee_{i=1}^s C_{1,i} \implies \bigwedge_{j=1}^t \neg C_{2,j}$.

This shows that $\forall I \forall X (C(X, I) \implies \exists p D(X, p, I))$.

Conversely, consider any assignment $\sigma' : X \cup \{p\} \cup I \rightarrow \{0, 1\}$ that causes D to evaluate to 1. Let $\sigma : X \cup I \rightarrow \{0, 1\}$ be the projection of σ' on $X \cup I$. By definition of circuit D , the function represented by the circuit $D|_{p=1, \neg p=1}$ is semantically equivalent to the function represented by C . Since all internal gates in D , viz. $\wedge$ and $\vee$ gates, are monotone, and since σ (in which either p or $\neg p$ is 0) causes D to evaluate to 1, by monotonicity, σ must cause $D|_{p=1, \neg p=1}$ to evaluate to 1 as well. Hence, σ causes $C(X, I)$ to evaluate to 1. This shows that $\forall I \forall X \forall p (D(X, p, I) \implies C(X, I))$. Hence $C \rightsquigarrow D$ holds. $\square$

Theorem 3. *Given a specification $C(X, I)$ in SAUNF, Algorithm $SkGen$ generates a polynomial-sized Skolem function vector in polynomial-time.*

Proof. The fact that Algorithm $SkGen$ generates a correct Skolem function vector follows from an inductive application of Theorem 2. Specifically, for each level i of recursion, if the Skolem function Ψ_D returned in line 9 by the $i + 1^{th}$ recursive call of $SkGen$ is correct for D , Theorem 2 ensures that the Skolem function Ψ_C computed in lines 10 and 11 of the i^{th} recursive call is correct for C .

To see why the terminating case of this recursion yields correct Skolem functions, note that when the recursion level is $k + 1$ (lines 1-2 of Algorithm 4), by Definition 3, the function represented by C is semantically independent of X . Hence any Skolem function vector for X suffices in line 2 of Algorithm 4.

To prove the polynomial bound on running time and hence, on size, of the generated Skolem function vector, we assume that the vector of functions used in line 2 of Algorithm 4 has size in $\mathcal{O}(k^2 \cdot |C|)$. Since an arbitrary vector of functions of I suffices in line 2, we can choose a $|X|$ -dimensional constant function vector, say $(0, \dots, 0)$

as the output of $GetAnyFuncVec$. Hence, the above assumption can *always* be satisfied.

Algorithm $SkGen$ has exactly $k + 1$ recursive calls, and in each of the first k calls, the steps in lines 4, 5, 6, 7 and 8 take time polynomial in $|C|$ and generate circuits E and D that are of size in $\mathcal{O}(|C|)$. Indeed, the circuit D in each recursion level $\leq k$ is simply C with the literal labels of some of its leaves replaced by other literals or by Boolean constants (possibly followed by simplification by constant propagation). Therefore, $|D| \leq |C|$ in each recursion level $\leq k$. The circuit E is similarly obtained by replacing some leaves of C with Boolean constants, other literals or conjunctions of two literals. Therefore, $|E| \leq 2 \times |C|$ in each recursion level $\leq k$. In the $k + 1^{th}$ recursive call, line 2 is executed, and as discussed above, we restrict it to take time in $\mathcal{O}(k^2 \cdot |C|)$. This also ensures that the size of the Skolem function vector returned in line 2 is in $\mathcal{O}(k^2 \cdot |C|)$.

Once the recursive calls start returning, lines 10 and 11 of Algorithm $SkGen$ are executed. Note that in line 10, the Skolem function for ℓ is obtained by feeding the inputs of circuit E (as obtained in the current level of recursion) by outputs of Skolem functions computed in later (or higher) levels of the recursion. We have already seen above that $|E|$ is at most $2 \times |C|$ in each recursion level $\leq k$. A Skolem function computed at recursion level j can potentially feed into E at all recursion levels in $\{1, \dots, j - 1\}$. Therefore, a maximum of $\sum_{j=2}^{k+1} (j - 1) = \mathcal{O}(k^2)$ connections may need to be created between the output of a Skolem functions generated at some recursion level and the input of E at a lower level of recursion. Therefore, constructing the entire Skolem function vector at recursion level 1 requires time (and, hence space) in $\mathcal{O}(k^2 \cdot |C|)$. $\square$

Lemma 5. *Let $\Psi(I)$ be a Skolem function vector for X in $C(X, I)$. Define $G(I)$ to be $C(\Psi(I), I)$, $\tilde{F}(X, I)$ to be $\bigwedge_{i=1}^n ((x_i \wedge \psi_i) \vee (\neg x_i \wedge \neg \psi_i))$, and $H(X, I)$ to be $(\tilde{F}(X, I) \vee C(X, I)) \wedge G(I)$. Then $H(X, I)$ is in SAUNF and both $H(X, I)$ and $C(X, I)$ represent semantically equivalent specifications.*

Proof. We first show that $C(X, I) \iff H(X, I)$. This involves showing two implications.

- $H(X, I) \Rightarrow C(X, I)$: We know from the definitions that $G(I) \Leftrightarrow C(\Psi(I), I) \Rightarrow \forall X (\tilde{F}(X, I) \Rightarrow C(X, I))$. The last implication follows from the observation that $\tilde{F}(X, I)$ simply asserts that $\bigwedge_{i=1}^n (x_i \Leftrightarrow \psi_i(I))$, and hence $C(\Psi(I), I) \Rightarrow \forall X (\tilde{F}(X, I) \Rightarrow C(X, I))$. We also know from the definition of $H(X, I)$ that $H(X, I) \Rightarrow G(I)$ and $H(X, I) \Rightarrow (\tilde{F}(X, I) \vee C(X, I))$. However, since $G(I) \Rightarrow \forall X (\tilde{F}(X, I) \Rightarrow C(X, I))$, it follows that $H(X, I) \Rightarrow (G(I) \wedge C(X, I))$. Hence $H(X, I) \Rightarrow C(X, I)$.
- $C(X, I) \Rightarrow H(X, I)$: We know that $C(X, I) \Rightarrow \exists X C(X, I) \Leftrightarrow C(\Psi(I), I) \Leftrightarrow G(I)$ by definition. It

follows that $C(X, I) \Rightarrow (C(X, I) \wedge G(I))$. However, from the definition of $H(X, I)$, we know that $(C(X, I) \wedge G(I)) \Rightarrow H(X, I)$. Hence, $C(X, I) \Rightarrow H(X, I)$.

To show that $H(X, I)$ is in SAUNF, note that $\tilde{F}(X, I) \equiv \bigwedge_{i=1}^n ((x_i \wedge \psi_i) \vee (\neg x_i \wedge \neg \psi_i))$ can be represented as a SAUNF circuit. Specifically, we represent $\bigwedge_{i=1}^n ((x_i \wedge \psi_i) \vee (\neg x_i \wedge \neg \psi_i))$ in the natural way as a 3-level circuit with an $\wedge$ -gate at the root having n children, each labeled $\vee$. The i^{th} child of the root has two children labeled $\wedge$ and representing the subfunctions $(x_i \wedge \psi_i(I))$ and $(\neg x_i \wedge \neg \psi_i(I))$ respectively. It is easy to see that every x_i (and $\neg x_i$) is $\wedge$ -unrealizable in this circuit. Hence, the circuit is in SAUNF w.r.t. X and the sequence of literal-consistent subsets of leaves $S = (S_1, S_2, \dots, S_{2n})$, where S_{2i-1} is the set of all leaves labeled x_i , and S_{2i} is the set of all leaves labeled $\neg x_i$, for $1 \leq i \leq n$.

By Lemma 7, we also know that $\tilde{F}(X, I) \mid_{x_1=1, \neg x_1=1, \dots, x_n=1, \neg x_n=1} \Leftrightarrow \exists X \tilde{F}(X, I) \Leftrightarrow 1$. The last equivalence follows from the definition of $\tilde{F}(X, I)$; specifically $\tilde{F}(\Psi(I), I) = 1$ for all values of I . It now follows from Definition 3 that the circuit $(\tilde{F}(X, I) \vee C(X, I)) \wedge G(I)$ is in SAUNF w.r.t. X and the sequence of literal-consistent subsets of leaves $S = (S_1, S_2, \dots, S_{2n})$ described above. To see why this is so, let R denote the circuit $(\tilde{F}(X, I) \vee C(X, I)) \wedge G(I)$. Since each S_i is $\wedge$ -unrealizable in $\tilde{F}(X, I)$, since $\tilde{F}(X, I)$ is combined with $C(X, I)$ through an $\vee$ -gate, and since $G(I)$ is semantically independent of X , the first 3 conditions of Definition 3 are easily seen to be satisfied. To check if the fourth condition also holds, note that $R \mid_{S_1:1, S_2:1, \dots, S_{2n}:1}$ represents $(\tilde{F}(X, I) \mid_{x_1=1, \neg x_1=1, \dots, x_n=1, \neg x_n=1} \vee C(X, I)) \wedge G(I)$. However, this simplifies to $(1 \vee C(X, I)) \wedge G(I) \Leftrightarrow G(I)$, which is semantically independent of X . This shows that condition 4 of Definition 3 holds, and hence R is in SAUNF w.r.t. X and the sequence $S = (S_1, S_2, \dots, S_{2n})$.

□

Theorem 4. Given a relational specification $C(X, I)$,

1. A Skolem function vector can be computed in polynomial time iff a semantically equivalent SAUNF form can be computed in polynomial time.
2. A Skolem function vector has polynomial size iff there exists a semantically equivalent SAUNF form of the specification that is polynomial sized.

Proof. 1. By Theorem 3, we can generate a Skolem function vector from a specification represented in SAUNF in time polynomial in the size of the representation. Therefore, if a semantically equivalent SAUNF form of the specification can be computed in polynomial time, we can obtain a Skolem function vector in polynomial time. In the reverse direction,

if a Skolem function vector $\Psi(I)$ can be computed in time polynomial in the size of the specification, we can construct the semantically equivalent specification $H(X, I) = (\tilde{F}(X, I) \vee C(X, I)) \wedge G(I)$, which by Lemma 5, is in SAUNF. Clearly, $G(Y) = C(\Psi(I), I)$ and $\tilde{F}(X, I)$ can be obtained in polynomial time if the Skolem function vector $\Psi(I)$ can be computed in polynomial time. This proves the first part of the theorem.

2. By Theorem 3, we can generate a polynomial sized Skolem function vector from a polynomial sized SAUNF form. In the reverse direction, we consider the semantically equivalent form $H(X, I) = (\tilde{F}(X, I) \vee C(X, I)) \wedge G(I)$, which by Lemma 5, is already in SAUNF. Note that $G(Y) = C(\Psi(I), I)$ and $\tilde{F}(X, I)$ are both polynomial-sized given a polynomial-sized Skolem function vector. □

Lemma 8. Suppose $G(X, I)$ is in SAUNF w.r.t X and a sequence S^G , and $H(X, I)$ is in SAUNF w.r.t. X and S^H . If there is no literal ℓ over X such that G has an ℓ -labeled leaf and H has a $\neg \ell$ -labeled leaf, combining the outputs of G and H by an $\wedge$ -gate gives a circuit in SAUNF w.r.t. X and (S^G, S^H) . Otherwise, an SAUNF circuit semantically equivalent to $G \wedge H$ cannot be constructed in time polynomial in $|G|, |H|$ unless $P = NP$. Such a circuit cannot have size polynomial in $|G|, |H|$ unless $\Pi_2^P = \Sigma_2^P$ (i.e. the polynomial hierarchy collapses to the second level).

Proof. Since there is no literal ℓ over X such that G has an ℓ -labeled leaf and H has a $\neg \ell$ -labeled leaf, it is easy to see that a leaf of G and a leaf of H cannot participate together in making a literal $\wedge$ -realizable in $G \wedge H$. Since G is in SAUNF w.r.t. X and S^G , and H is in SAUNF w.r.t. X and S^H , it then follows that $G \wedge H$ is also in SAUNF w.r.t. X and the sequence (S^G, S^H) (or alternatively, (S^H, S^G)).

The proof for the remainder of the lemma involves certain definitions of transformations.

Definition 6. For a circuit $F(X)$ with variable set $x_1, x_2, \dots, x_n$, we define $F^-(X, X')$ as the circuit obtained by replacing each instance of $\neg x_i$ at all leaves with a corresponding new variable x'_i for all i .

Claim 1. For any circuit $F(X)$, $F^-(X, X')$ is in SAUNF w.r.t. $X \cup X'$ for any sequence of literal-consistent subsets of leaves.

Proof. For any $F^-(X, X')$, there is no literal label of a leaf, whose complement is also the label of some other leaf in the circuit. This completely eliminates the possibility of subset of literal-consistent leaves being $\wedge$ -realizable in $F^-(X, X')$. □

Definition 7. For any circuit $F(X)$ with variable $x_1, x_2, \dots, x_n$, $F'(X, X')$ is defined as $\bigwedge_{i=1}^n ((x'_i \wedge \neg x_i) \vee$

1053 $(\neg x'_i \wedge x_i))$ where x'_i is the same variable introduced in 1104
 1054 $F^-(X, X')$. 1105

1055 Note that $F'(X, X')$ is equivalent to $\bigwedge_{i=1}^n (x'_i \Leftrightarrow \neg x_i)$ 1106
 1107

1056 **Claim 2.** For any circuit $F(X)$, $F'(X, X')$ is in 1109
 1057 SAUNF w.r.t. $X \cup X'$ and the sequence of literal- 1110
 1058 consistent leaves $S = (S_1, \dots, S_{2n}, S'_1, \dots, S'_{2n})$, where 1111
 1059 S_{2i-1} (resp. S'_{2i-1}) is the set of all leaves labeled x_i 1112
 1060 (resp. x'_i), and S_{2i} (resp. S'_{2i}) is the set of all leaves 1113
 1061 labeled $\neg x_i$ (resp. $\neg x'_i$) in $F'(X, X')$. 1114
 1115

1062 *Proof.* It is easy to see that for the sequence 1116
 1063 $(S_1, S_2, \dots, S_{2n})$, the first three conditions of Definition 1117
 1064 3 are satisfied. Moreover $F'|_{S_1:1, S_2:1, \dots, S_{2n}:1} = 1$. 1118
 1065 Hence, the circuit is in SAUNF w.r.t. $X \cup X'$ and 1119
 1066 $(S_1, \dots, S_{2n})$, and hence also w.r.t. $X \cup X'$ and the se- 1120
 1067 quence $S = (S_1, \dots, S_{2n}, S'_1, \dots, S'_{2n})$ as well. $\square$ 1121
 1122

1068 Note that it can be seen that $F^- \wedge F'$ is equisatisfiable 1123
 1069 with F . For an input F to the SAT problem, consider 1124
 1070 the conjunction of F^- , F' , both of which are in SAUNF 1125
 1071 by Claims 1, 2. 1126
 1127

- 1072 1. Consider the circuit $F^- \wedge F'$, where $X \cup X'$ is the 1128
 1073 set of outputs, and there are no inputs I . Suppose 1129
 1074 this circuit can be converted to a SAUNF circuit in 1130
 1075 P-time. Since a SAUNF circuit yields Skolem func- 1131
 1076 tions for all outputs in P-time, we can compute a 1132
 1077 Skolem function for every output (i.e. elements of 1133
 1078 $X \cup X'$) of $F^- \wedge F'$ in P-time. Since there are no in- 1134
 1079 puts, each of these Skolem functions must simplify 1135
 1080 down to a Boolean constant. If the given Skolem 1136
 1081 functions (Boolean constants for values of outputs 1137
 1082 in $X \cup X'$) satisfy F , the problem $F^- \wedge F'$ is satisfi- 1138
 1083 able. Since $F^- \wedge F'$ is equisatisfiable to F , this will 1139
 1084 solve satisfiability of F in P-time. Therefore we will 1140
 1085 have $P=NP$, if such a SAUNF circuit can be gener- 1141
 1086 ated in P-time. 1142
- 1087 2. If an equivalent poly-sized SAUNF circuit exists, the 1143
 1088 synthesized Skolem functions will also be polynomial 1144
 1089 in size. By the same argument as above, $F^- \wedge F'$ must 1145
 1090 be in P/Poly in this case, and by extension, the SAT 1146
 1091 problem will be in P/Poly. This will show that NP 1147
 1092 is contained in P/Poly. By Karp-Lipton Theorem, 1148
 1093 however, we know that NP is not in P/Poly unless the 1149
 1094 polynomial hierarchy collapses to the second level, i.e. 1150
 1095 unless $\Pi_2^P = \Sigma_2^P$. 1151
 1152

1096 $\square$ 1153
 1154

1097 **Theorem 5.1.** Given $C(X, I)$ and a sequence of 1155
 1098 literal-consistent disjoint subsets of leaves $S =$ 1156
 1099 $(S_1, \dots, S_k)$, checking if C is in SAUNF w.r.t. X and 1157
 1100 S is co-NP complete. 1158
 1159

- 1101 2. Given $C(X, I)$, checking if C is in SAUNF w.r.t. X 1160
 1102 and some (unspecified) sequence of subsets of leaves 1161
 1103 is co-NP hard and in Σ_2^P . 1162

Proof. 1. First, we show that identifying whether a given circuit is in SAUNF for a given sequence of subsets of leaves is in coNP. This is equivalent to asking whether the complement problem, i.e. if the given circuit is *not* in SAUNF for the given sequence of subsets of leaves, is in NP. We will define a non-deterministic polynomial-time machine M that solves this complement problem. The machine M first checks whether the input circuit (say C) is in NNF. This can be done by checking in each internal node of C is labeled either $\wedge$ or $\vee$, and if all negations (if any) are on the labels of leaves. Clearly, this check can be done in time polynomial in the size of C . If the circuit is found to be not in NNF, the machine M accepts, since C cannot be in SAUNF in this case. Otherwise (i.e. if C is in NNF), the machine M non-deterministically chooses a subset S_i of literal-consistent leaves in the given sequence S and executes the following operations.

Suppose the literal labeling leaves in the subset S_i is ℓ_i . The machine M (a) constructs the circuit $C' = C|_{S_1:1, \dots, S_{i-1}:1}$, (b) sets all leaves of C' that are not in S_i but are labeled ℓ_i to 0, (c) replaces all remaining labels ℓ on leaves by w and all labels $\neg \ell$ on leaves by w' , (d) guesses an assignment σ to all variables other than w and w' labeling leaves in the resulting circuit, and (e) checks if the resulting circuit represents the Boolean function $w \wedge w'$ for the assignment σ to other variables. Note that after step d, the resulting circuit represents a function of only w and w' (at most). Hence the check in step (e) can be performed by setting (w, w') to each of $(1, 1)$, $(1, 0)$, $(0, 1)$ and $(0, 0)$ and checking if the resulting circuit evaluates to 1, 0, 0 and 0 respectively. Clearly, all the steps from (a) through (e) above can be done in time polynomial in the size of the circuit C . If after step (e), the resulting circuit is found to represent $w \wedge w'$, then machine M accepts. In this case, C is not in SAUNF w.r.t. X and the given sequence S of subsets of leaves. Conversely, if the circuit is not in SAUNF w.r.t. X and the given sequence S , then there is a subset S_i of leaves in S that is $\wedge$ -realizable in C for the assignment described above. Hence the problem of identifying whether a circuit is *not* in SAUNF for a given sequence of subsets of leaves is in NP. Therefore, the problem of identifying whether C is in SAUNF for a given sequence of subsets of leaves is in co-NP.

Next, we show that the problem is co-NP hard. We reduce the problem of identifying whether a propositional formula represented by a CNF circuit is unsatisfiable to identifying whether an appropriately constructed circuit is in SAUNF for a specific sequence of subsets of literal-consistent leaves. For this reduction, we consider the specification represented by the circuit $C \wedge x \wedge \neg x$, where x is the sole output of the specification, and the inputs are the variables labeling leaves of C . Since there is only one output variable, there are only two (equivalent) ordering of subsets of

leaves labeled by output literals. It is easy to see that x (equivalently, $\neg x$) is $\wedge$ -realizable if and only if C is satisfiable. Hence, identifying whether a problem is in SAUNF for a given sequence of subsets of leaves is coNP-hard.

2. To prove the second part of the theorem, we use the following result from the polynomial hierarchy: $\Sigma_2^P = NP^{NP}$.

Specifically, we show that checking whether a given circuit C is in SAUNF w.r.t. some (unspecified) sequence of subsets of output literal-consistent leaves can be solved by an non-deterministic polynomial-time Turing machine M that invokes an NP oracle, i.e. $\in NP^{NP}$. Given $C(X, I)$, the machine M operates as follows.

- (a) Guesses an sequence S of subsets of literal-consistent leaves.
- (b) Reduces the problem of deciding whether $C(X, I)$ is not in SAUNF w.r.t X and the sequence S to checking the satisfiability of an appropriately constructed propositional formula φ . This reduction is similar to what we discussed above in the proof of part (1).
- (c) Feeds φ to the NP oracle
- (d) Accepts if and only if the NP oracle rejects

It follows that M accepts if and only if $\exists$ an sequence S of subsets of output literal-consistent leaves for which C is in SAUNF w.r.t. X and S . Hence, we have proved that our problem is contained in NP^{NP} .

The proof that the problem is co-NP hard is almost the same as the corresponding proof with a given sequence of literal-consistent subsets of leaves. We reduce the problem of identifying whether a formula represented by a CNF circuit is unsatisfiable to identifying whether the circuit is in SAUNF for some sequence of subsets of literal-consistent leaves. However, recall from the co-NP-hardness proof of part (1) that there are only two (equivalent) sequences of subsets of output-literal consistent leaves of the circuit $C \wedge x \wedge \neg x$, where C is a CNF circuit representing a propositional formula whose unsatisfiability we wish to check. Hence $C \wedge x \wedge \neg x$ is in SAUNF w.r.t. some (there are only two possibilities in our case) sequence of subsets of output literal-consistent leaves iff C is unsatisfiable. This proves co-NP-hardness of the problem. $\square$

Theorem 6. For $i \leq j \leq 2n, j - i < n, R[i, j]$ is representable by a polynomial (in n) sized SAUNF circuit.

Proof. By Theorem 4, we can generate a polynomial-sized SAUNF circuit in polynomial time given an explicit polynomial-sized Skolem function vector. Therefore, we focus on obtaining an explicit polynomial-sized Skolem function vector for $R[i, j]$, $i \leq j \leq 2n, j - i < n$. We use the notation X_i to denote the i^{th} least significant bit of X and $X_{i,j}$ denotes the bit-slice of X

from bits i to j (both included). We use similar notations for any other bit vectors Y, I . Now, we give explicit polynomial-sized Skolem function vectors for $R[i, j]$, $i \leq j \leq 2n, j - i < n$. We consider the following two cases:

- $i \leq n$: The Skolem function vector for X, Y is given by

$$\Psi_{X_k} = \begin{cases} 0 & \text{if } k \neq i - 1 \\ 1 & \text{if } k = i - 1 \end{cases}$$

$$\Psi_{Y_k} = \begin{cases} I_{k+i-1} & \text{if } k \leq j + 1 - i \\ 0 & \text{if } k > j + 1 - i \end{cases}$$

Intuitively, $X = 2^{i-1}, Y = I_{i,j}$. Clearly this results in the matching of bits i to j as required.

- $i > n$: Following a similar logic as above, the Skolem function vector for X, Y is given by

$$\Psi_{X_k} = \begin{cases} 0 & \text{if } k \neq n \\ 1 & \text{if } k = n \end{cases}$$

$$\Psi_{Y_k} = \begin{cases} 0 & \text{if } k \leq i - n \\ I_{k+n-1} & \text{if } k > i - n, k \leq j + 1 - n \\ 0 & \text{if } k > j + 1 - n \end{cases}$$

Intuitively, $X = 2^{n-1}, Y = 2^{i-n} \times I_{i,j}$. Clearly this results in the matching of bits i to j .

Having generated a Skolem function vector for each specification $R[i, j], j - i < n$, we can generate a corresponding SAUNF circuit. The size of the Skolem function vector is clearly polynomial in n . By Theorem 4, this proves the theorem. $\square$

Theorem 7. The relation $X = I/Y$, with inputs I, Y restricted to odd numbers (i.e. the relation evaluates to 0 if I or Y is even), is representable as a polynomial (in n) sized SAUNF circuit.

Proof. We give a skolem function vector for division, which will generate a SAUNF form by Theorem 4. Now, suppose we have inputs I, Y and we have to find X such that $X \times Y = I$. We first show that for odd inputs, if $(X \times Y) \bmod 2^n = I \bmod 2^n$ and if I is divisible by Y then $X \times Y = I$. Suppose there are two values X_1, X_2 such that $(X_1 \times Y) \bmod 2^n = I \bmod 2^n$ and $(X_2 \times Y) \bmod 2^n = I \bmod 2^n$. Then $(X_1 - X_2) \times Y \equiv 0 \bmod 2^n$. However, Y is odd, and therefore co-prime to 2^n , $(X_1 - X_2) \equiv 0 \bmod 2^n$. Since, $X_1, X_2 < 2^n$, $X_1 = X_2$. Therefore the generated X from the skolem function is correct if there exists a solution that matches the least significant n bits of I . Using the notation defined above, denote X_i to be the i^{th} least significant bit of X and $X_{i,j}$ to denote the bit vector from X_i to X_j (both included). Now, since the input is odd, $Y_1 = 1, I_1 = 1$. Therefore $\Psi_{X_1} = 1$.

Now note that $(X \times Y)_i = (X_{1,i} \times Y_{1,i})_i = (X_{1,i-1} \times Y_{1,i} + X_i \cdot Y_1 \times 2^{i-1})_i = X_i \cdot Y_1 \oplus (X_{1,i-1} \times Y_{1,i})_i$ (using the structure of multiplication). Therefore $I_i = X_i \oplus (X_{1,i-1} \times Y_{1,i})_i$. Therefore, $X_i = I_i \oplus (X_{1,i-1} \times Y_{1,i})_i$.

Therefore, given the skolem function for $\Psi_{X_1} = 1$ to $\Psi_{X_{i-1}}$, we recursively generate the skolem function for X_1 , giving us the skolem function vector for X . $\square$

Note that while we use particular skolem functions in generating the SAUNF form, the form can generate other skolem functions as well (from Algorithm 4).

Guarantees in Algorithms 2, 3: We formally give statements on Algorithms 2, 3 and give the proofs, elaborating on the general description given.

Lemma 9. *Algorithm 2 returns an $\wedge$ -unrealizable subset of ℓ -leaves and takes a worst-case exponential (in $|C|$) time.*

Proof. First, we show that it always returns an $\wedge$ -unrealizable subset. We give a proof by contradiction. Suppose that the set T is $\wedge$ -realizable for an assignment σ' in C ($T = \text{GetClausesEvaluatingToL}(C, \sigma', \ell)$). We show that D will also be $\wedge$ -realizable for ℓ . Under the assignment σ' , all clauses that contain leaves in HitS evaluate to 1 irrespective of ℓ as $\text{HitS} \cap T = \emptyset$ and a clause containing ℓ can evaluate to only ℓ or 1. Therefore, for all $S \in \text{AllS}$, $\text{DisjoinWithoutLit}(S, \ell)$ evaluates to 1 as at the clause corresponding to the leave in HitS evaluates to 1. Therefore, for an assignment σ' , D will also be $\wedge$ -realizable if C is also $\wedge$ -realizable for the same assignment. However, this is a contradiction to the breaking condition of the loop that D is not $\wedge$ -realizable.

Now, we look at the worst case running time of the Algorithm. Suppose that a set S is pushed into AllS . Now, D is conjuncted with $\text{DisjoinWithoutLit}(S, \ell)$. For any assignment σ' , if $S = \text{GetClausesEvaluatingToL}(C, \sigma', \ell)$, then $\text{DisjoinWithoutLit}(S, \ell)$ evaluates to 0. Therefore, any set $\text{CurrS} \neq S$ at line 5 as D cannot be $\wedge$ -realizable for such an assignment. Therefore, the number of iterations is the number of subsets of ℓ -leaves in C , which is exponential in $|C|$. Each iteration involves a SAT call. Therefore, the worst case complexity is exponential in $|C|$.

However, note that if we can modify the breaking condition of the loop to involve a TimeOut. In such a case, we return the $\emptyset$. This reduces the worst case complexity, providing a tradeoff between the running time and precision of the algorithm. $\square$

We now state the theorem associated with Algorithm 3.

Lemma 10. *Algorithm 3 returns a SAUNF circuit.*

Proof. We follow a proof similar to the argument given in Section 6. In the circuit F constructed in line 11 of Algorithm 3, the set containing only the leaf ℓ (call it L_1) conjoined with C_1 is $\wedge$ -unrealizable as it meets up $\neg\ell$ at an $\vee$ gate. In $F_{|L_1:1}$, we can show that set $S^{C'}$ of ℓ -leaves of C' is $\wedge$ -unrealizable as it was already $\wedge$ -unrealizable in the circuit C .

For $F_{|L_1:1}$ to be $\wedge$ -realizable for an assignment C' should evaluate to ℓ , C_1 to 0 and C_2 to 1. We show that if such a case is possible for an assignment, then S would have been $\wedge$ -realizable in C for that assignment, which is not possible by Lemma 9. We divide the clauses in C into 4 sets: (i) Clauses in S , (ii) Clauses that have positive ℓ -leaves not in S , (iii) Clauses with the negative literals $\neg\ell$ (iv) Clauses that have no dependence on ℓ . We look at assignments to each of these clauses in C for the particular assignment. C' represents the circuit for clauses in (i) which evaluates to ℓ for $\wedge$ -realizability in $F_{|L_1:1}$. Since C_2 evaluates to 1, all clauses in C_2 evaluate to 1. Therefore, the clauses in which $\ell = 0$ (C_2 contains only those clauses with ℓ -leaves not in S) or have no dependence to ℓ evaluate to 1. Therefore, circuits corresponding to clauses in (ii) and (iv) evaluate to 1. Since (iv) evaluates to 1, but C_1 is 0, at least one with $\neg\ell$ set to 0 evaluates to 0. Therefore, in circuit C , it will evaluate to $\neg\ell$ and therefore, circuit correspond to (iii) evaluates to $\neg\ell$. This shows that C would have been $\wedge$ -realizable in C , which is not possible by Lemma 9. We set $S^{C'}$ of ℓ -leaves of C' to 1.

In $F_{|L_1:1, S^{C':1}|}$, all positive leaves of ℓ have been set to 1, and therefore we can set the only $\neg\ell$ (denote $L' = \{\neg\ell\}$) to 1. $F_{|L_1:1, S^{C':1}, L':1|} = C_1 \vee C_2$, which is in SAUNF assuming the recursive calls return correct representations and using Lemma 6. The base condition is when C is independent of X , for which it already returns the correct value. $\square$

This algorithm represents a significant reduction in size over Shannon expansion, as we expand the circuit D obtained after setting certain clauses of C to 1 (line 6 of Algorithm 3). In the worst-case, however, this reduces to a Shannon-expansion based conversion of C .

A Pseudocode

Algorithm 4: SkGen(C, S, r)

Input: $C(X, Y)$: Relational spec, $\Psi_C(Y)$:
Skolem function for X in C ; $n = |X|$
 r : Recursion level
Output: $\Psi_C(Y)$: Skolem function vector for C

```
1 SF = GetSAUNF( $\Psi$ ,  $C$ );
2 if  $r = 2n$  then
3    $\Psi_C(Y) := \neg \Psi_C(Y)$ ;
4 else
5    $S := \wedge$ -unrealizable Literal label of leaves ;
6    $D := \text{CPropSimp}(C \upharpoonright_{S=1})$ ;
   // CPropSimp propagates constants and
   // eliminates gates with constant
   // outputs
7    $\Psi_D(Y) = \text{SkGen}(D, S, r + 1)$ ;
8    $temp :=$ 
   CPropSimp( $E(\Psi_D(Y), \ell = \neg \psi(Y), Y)$ );
9    $\psi_l(Y) = (\neg \psi(Y) \wedge temp) \vee (\psi(Y) \wedge \neg temp)$  ;
10 return  $\Psi_C(Y)$ ;
```
